4. Сверточные нейронные сети. Классификация изображений.

Цель работы - реализовать и исследовать сверточную нейронную сеть (CNN) для задачи классификации изображений на выборке MNIST. В рамках работы требуется построить модель на основе двух типов сверточных блоков: (a) {Conv2d–ReLU–MaxPool} и (b) {Conv2d–ReLU–Conv2d–ReLU–MaxPool}, а затем подобрать сочетание части гиперпараметров, обеспечивающее наилучшее качество классификации. В качестве варьируемых параметров рассматриваются: коэффициент обучения, тип и количество сверточных блоков, параметры свертки (kernel_size, stride) и параметры пулинга (kernel_size, stride). Дополнительно необходимо оценить минимальное число эпох обучения, достаточное для достижения точности классификации (accuracy) не ниже 90% для выбранной конфигурации сети.

Задания:

1. Реализация и последующее обучение сверточной нейронной сети с подбором гиперпараметров
2. Оценка количества эпох, достаточное для классификации не меньшей “90%”

Ссылка на исходный код лабораторной работы:
https://github.com/F1ameX/Modern-Methods-of-Deep-Machine-Learning/blob/main/4_CNN/4_CNN.ipynb

Ход работы
Реализация и обучение CNN с подбором гиперпараметров

В первой части работы была реализована сверточная нейронная сеть (CNN) для классификации изображений датасета MNIST. Сеть строилась из повторяющихся сверточных блоков двух типов:

1. A = {Conv2d–ReLU–MaxPool}
2. B = {Conv2d–ReLU–Conv2d–ReLU–MaxPool}

Число блоков в нейросети задавалось отдельным гиперпараметром.

```python
class ConvBlockA(nn.Module):
    def __init__(
        self,
        input_channels: int,
        output_channels: int,
        conv_kernel_size: int,
        conv_stride: int,
        pool_kernel_size: int,
        pool_stride: int,
    ):
        super().__init__()

        self.conv = nn.Conv2d(
            in_channels=input_channels,
            out_channels=output_channels,
            kernel_size=conv_kernel_size,
            stride=conv_stride,
            padding=conv_kernel_size // 2,
        )
        self.activation = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=pool_kernel_size, stride=pool_stride)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.conv(x)
        x = self.activation(x)
        x = self.pool(x)
        return x
```

Рисунок 1 - Сверточный блок типа А

```python
class ConvBlockB(nn.Module):
    def __init__(
        self,
        input_channels: int,
        output_channels: int,
        conv_kernel_size: int,
        conv_stride: int,
        pool_kernel_size: int,
        pool_stride: int,
    ):
        super().__init__()

        self.conv1 = nn.Conv2d(
            in_channels=input_channels,
            out_channels=output_channels,
            kernel_size=conv_kernel_size,
            stride=conv_stride,
            padding=conv_kernel_size // 2,
        )
        self.conv2 = nn.Conv2d(
            in_channels=output_channels,
            out_channels=output_channels,
            kernel_size=conv_kernel_size,
            stride=1,
            padding=conv_kernel_size // 2,
        )

        self.activation = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=pool_kernel_size, stride=pool_stride)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.conv1(x)
        x = self.activation(x)

        x = self.conv2(x)
        x = self.activation(x)

        x = self.pool(x)
        return x
```

Рисунок 2 - Сверточный блок типа B

После прохождения сверточной части карта признаков разворачивалась (Flatten) и подавалась в один полносвязный слой, формирующий логиты классов.

```python
class CNN(nn.Module):
    def __init__(
        self,
        block_type: str = "a",
        n_blocks: int = 2,
        base_channels: int = 16,
        conv_kernel_size: int = 3,
        conv_stride: int = 1,
        pool_kernel_size: int = 2,
        pool_stride: int = 2,
        dropout: float = 0.0,
        num_classes: int = 10,
        kernel_size: int | None = None,
        pool_kernel: int | None = None,
        base_channels_alias: int | None = None,
        n_blocks_alias: int | None = None,
        pool_stride_alias: int | None = None,
        conv_stride_alias: int | None = None,
    ):
        super().__init__()
        assert block_type in ("a", "b")
        assert n_blocks >= 1

        if kernel_size is not None:
            conv_kernel_size = kernel_size
        if pool_kernel is not None:
            pool_kernel_size = pool_kernel
        if base_channels_alias is not None:
            base_channels = base_channels_alias
        if n_blocks_alias is not None:
            n_blocks = n_blocks_alias
        if pool_stride_alias is not None:
            pool_stride = pool_stride_alias
        if conv_stride_alias is not None:
            conv_stride = conv_stride_alias

        BlockClass = ConvBlockA if block_type == "a" else ConvBlockB

        blocks = []
        input_channels = 1
        channels = base_channels
```

Рисунок 3 - Первая часть кода реализации CNN

```python
    for _ in range(n_blocks):
        blocks.append(
            BlockClass(
                input_channels=input_channels,
                output_channels=channels,
                conv_kernel_size=conv_kernel_size,
                conv_stride=conv_stride,
                pool_kernel_size=pool_kernel_size,
                pool_stride=pool_stride,
            )
        )
        input_channels = channels
        channels *= 2

    self.features = nn.Sequential(*blocks)

    with torch.no_grad():
        dummy_x = torch.zeros(1, 1, 28, 28)
        try:
            dummy_out = self.features(dummy_x)
        except RuntimeError as e:

            raise ValueError("Invalid geometry: conv/pool params collapse 28x28 to 0x0") from e
        flatten_dim = dummy_out.view(1, -1).shape[1]

    self.dropout = nn.Dropout(dropout) if dropout > 0 else nn.Identity()
    self.classifier = nn.Linear(flatten_dim, num_classes)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    x = self.features(x)
    x = x.flatten(1)
    x = self.dropout(x)
    logits = self.classifier(x)
    return logits
```

Рисунок 4 - Вторая часть кода реализации CNN

Далее были загружены данные MNIST и подготовлены DataLoader’ы для обучения и валидации.

```
SEED = 52
BATCH_SIZE = 128

torch.manual_seed(SEED)
np.random.seed(SEED)
random.seed(SEED)

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

full_train_dataset = datasets.MNIST(root="./data", train=True, download=True, transform=transform)
test_dataset = datasets.MNIST(root="./data", train=False, download=True, transform=transform)

train_size = 50_000
val_size = len(full_train_dataset) - train_size

train_dataset, val_dataset = random_split(
    full_train_dataset,
    [train_size, val_size],
    generator=torch.Generator().manual_seed(SEED)
)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=2, pin_memory=True)
val_loader   = DataLoader(val_dataset,   batch_size=BATCH_SIZE, shuffle=False, num_workers=2, pin_memory=True)
test_loader  = DataLoader(test_dataset,  batch_size=BATCH_SIZE, shuffle=False, num_workers=2, pin_memory=True)

len(train_dataset), len(val_dataset), len(test_dataset)
```

Рисунок 5 - Код получения датасета

```
Mini-train size: 3000
Mini-val size:   1000
Mini-train class counts: {0: 300, 1: 300, 2: 300, 3: 300, 4: 300, 5: 300, 6: 300, 7: 300, 8: 300, 9: 300}
Mini-val class counts:   {0: 100, 1: 100, 2: 100, 3: 100, 4: 100, 5: 100, 6: 100, 7: 100, 8: 100, 9: 100}
```

Рисунок 6 - Размерность мини-датасетов для обучения на Optuna

Для подбора гиперпараметров использовалась Optuna: перебирались коэффициент обучения, weight decay, тип блоков и их количество, параметры свертки (kernel_size, stride), параметры пулинга (kernel_size, stride), базовое число каналов и dropout. Качество оценивалось по accuracy на валидационной выборке после короткого обучения в несколько эпох.

```python
def objective(trial):
    learning_rate = trial.suggest_float("learning_rate", 1e-4, 5e-3, log=True)
    weight_decay  = trial.suggest_float("weight_decay", 0.0, 1e-3)
    block_type    = trial.suggest_categorical("block_type", ["a", "b"])
    n_blocks      = trial.suggest_int("n_blocks", 1, 3)
    kernel_size   = trial.suggest_categorical("kernel_size", [3, 5])
    conv_stride   = 1
    pool_kernel   = trial.suggest_categorical("pool_kernel", [2, 3])
    pool_stride   = 2
    base_channels = trial.suggest_categorical("base_channels", [8, 16, 32])
    dropout       = trial.suggest_float("dropout", 0.0, 0.4)

    try:
        model = CNN(
            block_type=block_type,
            n_blocks=n_blocks,
            base_channels=base_channels,
            kernel_size=kernel_size,
            conv_stride=conv_stride,
            pool_kernel=pool_kernel,
            pool_stride=pool_stride,
            dropout=dropout,
        )
    except (RuntimeError, ValueError):
        raise optuna.TrialPruned()

    _, val_acc = quick_train_eval(
        model,
        mini_train_loader,
        mini_val_loader,
        learning_rate=learning_rate,
        weight_decay=weight_decay,
        epochs=3,
    )

    return val_acc


study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=25, catch=(RuntimeError, ValueError))

print("best value (val_acc):", study.best_value)
print("best params:", study.best_params)
```

Рисунок 7 - Функция оптимизации гиперпараметров при помощи Optuna

По итогам оптимизации были получены лучшая конфигурация гиперпараметров и соответствующая точность. Дополнительно создан датафрейм с результатами всех прогонов (trial value и параметры).

	trial	val_acc	block	blocks	k	pool_k	base_ch	dropout	lr	wd
0	11	0.956	b	2	5	3	16	0.084370	0.001829	0.000990
1	14	0.953	b	2	5	3	16	0.055420	0.001482	0.000827
2	13	0.951	b	2	5	3	16	0.087916	0.001540	0.000990
3	10	0.951	b	2	5	3	16	0.086496	0.001761	0.000960
4	19	0.951	b	2	5	3	16	0.315285	0.001103	0.000462
5	22	0.949	b	2	5	3	16	0.004988	0.002256	0.000936
6	7	0.948	b	1	5	3	8	0.070398	0.003304	0.000655
7	24	0.946	b	2	5	3	16	0.114712	0.002043	0.000714
8	21	0.939	b	2	5	3	16	0.070558	0.001602	0.000908
9	18	0.938	b	2	5	3	32	0.039354	0.000517	0.000879
10	23	0.935	b	2	5	3	16	0.059062	0.001254	0.000809
11	12	0.935	b	2	5	3	16	0.108359	0.001602	0.000994
12	15	0.928	b	2	5	3	16	0.037346	0.001089	0.000787
13	8	0.922	a	1	5	2	8	0.232120	0.002984	0.000129
14	9	0.911	b	1	3	2	8	0.399765	0.004863	0.000040

Рисунок 8 - Таблица лучших итераций подбора гиперпараметров при помощи Optuna

Оценка количества эпох, достаточное для “хорошей” классификации

Во второй части работы для выбранной по Optuna конфигурации CNN была оценена минимальная длительность обучения, достаточная для достижения качества accuracy ≥ 90%. Модель обучалась на обучающей выборке MNIST, после каждой эпохи вычислялась accuracy на валидационной.

```
epoch=01 | train_loss=0.1729 | val_loss=0.0662 | val_acc=0.9788
epoch=02 | train_loss=0.0506 | val_loss=0.0441 | val_acc=0.9863
epoch=03 | train_loss=0.0412 | val_loss=0.0374 | val_acc=0.9884
epoch=04 | train_loss=0.0343 | val_loss=0.0523 | val_acc=0.9832
epoch=05 | train_loss=0.0312 | val_loss=0.0551 | val_acc=0.9829
epoch=06 | train_loss=0.0317 | val_loss=0.0458 | val_acc=0.9873
epoch=07 | train_loss=0.0280 | val_loss=0.0506 | val_acc=0.9855
epoch=08 | train_loss=0.0286 | val_loss=0.0392 | val_acc=0.9887
epoch=09 | train_loss=0.0276 | val_loss=0.0413 | val_acc=0.9862
epoch=10 | train_loss=0.0284 | val_loss=0.0283 | val_acc=0.9906
epoch=11 | train_loss=0.0263 | val_loss=0.0447 | val_acc=0.9860
epoch=12 | train_loss=0.0243 | val_loss=0.0458 | val_acc=0.9872
epoch=13 | train_loss=0.0247 | val_loss=0.0473 | val_acc=0.9860
epoch=14 | train_loss=0.0247 | val_loss=0.0379 | val_acc=0.9891
epoch=15 | train_loss=0.0250 | val_loss=0.0332 | val_acc=0.9897
epoch=16 | train_loss=0.0249 | val_loss=0.0433 | val_acc=0.9865
epoch=17 | train_loss=0.0234 | val_loss=0.0451 | val_acc=0.9863
epoch=18 | train_loss=0.0240 | val_loss=0.0324 | val_acc=0.9899
epoch=19 | train_loss=0.0219 | val_loss=0.0386 | val_acc=0.9879
epoch=20 | train_loss=0.0227 | val_loss=0.0331 | val_acc=0.9903

First epoch reaching val_acc >= 0.90: 1
Final test_acc: 0.9912
Final test_loss: 0.026322905584378168
```

Рисунок 9 - Обучение лучшей по Optuna конфигурации по эпохам

Выводы

По итогам работы была реализована сверточная нейронная сеть для MNIST с двумя вариантами повторяющихся блоков. Для ускорения подбора гиперпараметров была сформирована стратифицированная мини-выборка (с сохранением баланса классов), на которой проводилась оптимизация.

Подбор части гиперпараметров выполнялся с помощью Optuna, а качество оценивалось по accuracy на валидационной выборке после короткого обучения в несколько эпох. В результате Optuna нашла конфигурацию, дающую на мини-валидации val_acc = 0.962; в лучших решениях чаще встречались блоки типа B, увеличенное число базовых каналов и умеренный dropout, что указывает на пользу более “выразительного” сверточного блока при сохранении регуляризации.

Далее для выбранной конфигурации была оценена достаточная длительность обучения для достижения требуемого качества accuracy $\geq 90\%$. При обучении на полной обучающей выборке MNIST порог 0.90 по валидации был достигнут уже на 1-й эпохе, а дальнейшее обучение давало стабилизацию метрик без необходимости увеличивать число эпох. Итоговое качество на тестовой выборке составило test_acc = 0.9912 при test_loss $\approx$ 0.0263, что подтверждает корректность реализации сети и эффективность выбранного набора гиперпараметров.